EXPERIMENT 14

Exploring Pro-Code Extension of a Zoho Creator Application using Zoho Catalyst / Custom Functions

Name: Jobeson J | **Register Number:** 240701217
Subject: Low Code Application Development using Zoho Creator
Project: Bank Loan Application & Pre-Screening Management

Aim

To explore how pro-code capability can extend a low-code Zoho Creator application, by writing and calling a custom Deluge function in the Loan Application that computes the reducing-balance EMI, and by describing the role of Zoho Catalyst for heavier work.

Introduction

Experiment 5 used a simplified formula for the estimated EMI. A bank needs the standard reducing-balance instalment, which is a mathematical rule better written once as code than rebuilt in several formula fields. Zoho Creator supports this through **custom functions**: reusable Deluge units that take parameters and return a value, and that any workflow can call. Beyond that, Zoho Catalyst is Zoho's serverless platform for code in general-purpose languages; a Creator application can call such a service through a custom API when the work is too heavy or too specialised for Deluge.

The EMI Rule

For principal P, annual interest rate R (in percent) and tenure of n months, with monthly rate r = R / 12 / 100, the instalment is $\mathbf{EMI = P \times r \times (1 + r)^n / ((1 + r)^n - 1)}$. A worked check: for P = 15,00,000, R = 9.25 and n = 180, the EMI is about 15,438; for P = 5,00,000, R = 10.5 and n = 60, it is about 10,747. These two cases are used as test values.

Procedure

1. Open the application and go to **Workflow → Functions** (custom functions).
2. Create a new function named **Calculate_EMI** with the parameters principal, annualRate and months, returning a decimal.
3. Enter the script of Code 14.1 and save.
4. Open Loan_Application_Form's workflow and call the function from the on-success or on-edit action, as in Code 14.2.
5. Run the two test values of the EMI rule and compare the output with the expected figures.
6. Add a validation inside the function so that a zero or negative principal, rate or tenure returns 0 instead of an error.
7. Optional: describe in the observations how the same function could be moved to a Catalyst service and called through a custom API.

```
// Code 14.1 — Calculate_EMI custom function (reference implementation)
real Calculate_EMI(real principal, real annualRate, int months)
{
    if (principal <= 0 || annualRate <= 0 || months <= 0)
    {
        return 0;
    }
    r = annualRate / 12 / 100;
    factor = (1 + r).power(months);          // confirm .power() in the Deluge reference for your version
    emi = principal * r * factor / (factor - 1);
    return emi.round(2);
}
```

Code 14.1 — Custom function

```
// Code 14.2 — Calling the function from a form workflow
emi = thisapp.Calculate_EMI(input.Loan_Amount_Requested, input.Interest_Rate_Offered, 180);
input.Estimated_Monthly_EMI = emi;
info "EMI for " + input.Loan_Application_ID + " = " + emi;
```

Code 14.2 — Function call

Test Table

Table 14.1: Function test cases (observed column to be completed)

Principal	Rate (%)	Months	Expected EMI	Observed
15,00,000	9.25	180	about 15,438	
5,00,000	10.5	60	about 10,747	
0	9.25	180	0	
15,00,000	0	180	0	

Low-Code versus Pro-Code for this Task

Table 14.2: Where to place the logic

Aspect	Formula field	Custom function	Catalyst / external service
Reuse	Only in that form	Any workflow	Any application or system
Complexity	Simple arithmetic	Moderate logic	Large or specialised logic
Testing	Manual	Test through a workflow and logs	Unit tests in a code repository
Where it runs	Form	Creator (Deluge)	Separate serverless platform

Output

SCREENSHOT TO ATTACH

Calculate_EMI function in the Deluge editor

Fig 14.1 — Custom function

SCREENSHOT TO ATTACH

Workflow calling the function, and the execution log with the computed EMI for a test record

Fig 14.2 — Function call and log output

SCREENSHOT TO ATTACH

Loan_Application_Report showing the computed Estimated_Monthly_EMI

Fig 14.3 — Result in the report

Result

A reusable Deluge function for the reducing-balance EMI was written and called from the loan workflow, and the position of Zoho Catalyst as a further extension was described.

Conclusion

Custom functions are the first step from low-code towards pro-code: the same business rule is written once, tested against known values and reused. When logic outgrows Deluge, the same pattern continues with a Catalyst service behind a custom API.